

รายงาน

Lab 10: Spring WebFlux & WebClient

วิชา CP353002 หลักการออกแบบและพัฒนาซอฟต์แวร์

จัดทำโดย: นายพิสิษฐ์ ทรัพย์อุดมโชติ 673380285-2 Sec.1

GitHub Repository: https://github.com/phisit10/LAB10_673380285-2

1. Reactive Programming vs Blocking I/O

1.1 Blocking I/O คืออะไร

ในรูปแบบดั้งเดิม (Blocking / Synchronous) เมื่อ Controller เรียกใช้เมธอดที่ต้องรอผลลัพธ์ เช่น การอ่านฐานข้อมูลหรือเรียก API ภายนอก Thread ที่กำลังประมวลผล request นั้นจะ "หยุดรอ" (block) จนกว่าผลลัพธ์จะกลับมา ระหว่างที่รออยู่ Thread นั้นทำอะไรไม่ได้เลย และเซิร์ฟเวอร์ต้องเปิด Thread ใหม่สำหรับทุก request ที่เข้ามาพร้อมกัน (โมเดลแบบ 1 request : 1 thread ของ Tomcat)

1.2 Reactive Programming คืออะไร

Reactive Programming เป็นรูปแบบการเขียนโปรแกรมแบบ Asynchronous และ Non-blocking โดยใช้แนวคิด Reactive Streams — เมธอดจะไม่คืนค่าผลลัพธ์ทันที แต่คืน "ตัวแทนของผลลัพธ์ในอนาคต" (เช่น Mono หรือ Flux ใน Project Reactor ที่ Spring WebFlux ใช้) เมื่อข้อมูลพร้อม ระบบจะแจ้งกลับผ่านกลไก subscribe/callback แทนที่จะให้ Thread นั่งรอเฉยๆ Thread เดียวจึงสามารถสลับไปจัดการ request อื่นระหว่างรอ I/O ได้ ทำให้ใช้ทรัพยากรน้อยลงมากเมื่อมี concurrent request จำนวนมาก (Spring WebFlux รันบน Reactor Netty แทน Tomcat)

1.3 ตารางเปรียบเทียบ

	Blocking (แบบเดิม)	Reactive
การทำงานของ Thread	รอจนกว่าจะได้ผล	ไม่รอ — ไปทำงานอื่น
Scalability	1 thread per request	Few threads, many requests
Return Type	String result	Mono<String> result
Server ที่ใช้	Tomcat (Servlet)	Reactor Netty
การใช้งานที่เหมาะสม	CRUD ทั่วไป	High-concurrency, streaming

2. Mono vs Flux

2.1 Mono<T>

Mono<T> ใช้แทนผลลัพธ์แบบ asynchronous ที่มีค่าได้ "0 หรือ 1 ค่า" เท่านั้น เหมาะกับ operation ที่คืนค่าขึ้นเดียว หรือไม่มีค่าเลย ในโค้ดของ Lab นี้ใช้ Mono ใน 4 จุด:

- findById(id) / getById(id) — ค้นหาสินค้า 1 ชิ้น อาจเจอ (Mono.just) หรือไม่เจอ (Mono.empty)
- save(product) — บันทึกสินค้า 1 ชิ้น แล้วคืนสินค้าชิ้นนั้นกลับมา
- deleteById(id) — ลบสินค้า ไม่มีข้อมูลคืนกลับ จึงใช้ Mono<Void>
- getDiscountedPrice(id) — คำนวณราคาหลังลดของสินค้า 1 ชิ้น ได้ตัวเลขเดียว จึงใช้ Mono<Double>

2.2 Flux<T>

Flux<T> ใช้แทนผลลัพธ์แบบ asynchronous ที่มีค่าได้ "0 ถึงหลายค่า" (stream ของข้อมูล) เหมาะกับ operation ที่คืนค่าเป็น collection ในโค้ดของ Lab นี้ใช้ Flux ใน 2 จุด:

- findAll() / getAll() — คืนสินค้าทั้งหมดใน store ซึ่งมีได้หลายชิ้น
- findByCategory(category) / getCategory(category) — คืนสินค้าที่กรองตามหมวดหมู่ ซึ่งอาจมีได้หลายชิ้นเช่นกัน

2.3 เกณฑ์การเลือกใช้ (Rule of Thumb)

- ผลลัพธ์เป็น "หนึ่งชิ้น หรือ ไม่มี" → ใช้ Mono<T>
- ผลลัพธ์เป็น "หลายชิ้น" (list/collection) → ใช้ Flux<T>
- เป็นการกระทำ (action) ที่ไม่มี payload กลับมา เช่น delete → ใช้ Mono<Void>

เหตุผลที่ต้องแยกสองชนิดนี้แทนที่จะใช้ List<T> ธรรมดา คือ Mono และ Flux เป็นแบบ Non-blocking และ Lazy — จะไม่เริ่มทำงานจนกว่าจะมีการ subscribe (เช่น ตอนที่ WebFlux ส่ง response กลับให้ client) ทำให้สามารถ chain operation อย่าง .map(), .filter(), .flatMap() ได้โดยไม่ block thread ระหว่างรอข้อมูล

3. WebClient Method Chain

WebClient คือ HTTP Client แบบ Reactive ของ Spring ใช้แทน RestTemplate แบบเดิม (ซึ่งเป็น blocking) ตัวอย่าง method chain จากไฟล์ ProductWebClient.java:

```
client.get()           // 1. HTTP method
.uri("/products/{id}", id) // 2. กำหนด URL
.retrieve()           // 3. เริ่มรับ response
.bodyToMono(Product.class); // 4. แปลงเป็น Mono<T>
```

3.1 .get() / .post() / .delete()

เป็นจุดเริ่มต้นของการเรียก HTTP request แต่ละเมธอดจะสอดคล้องกับ HTTP method ที่ใช้จริง — .get() สำหรับดึงข้อมูล, .post() สำหรับส่งข้อมูลใหม่ (ต้องตามด้วย .bodyValue(...) เพื่อแนบ request body), .delete() สำหรับลบข้อมูล เมธอดเหล่านี้คืนค่าเป็น RequestHeadersUriSpec ที่ใช้ chain กันต่อไปได้

3.2 .uri(...)

กำหนด path ของ endpoint ที่จะเรียก โดยรองรับการใส่ path variable ผ่าน placeholder {id} แล้วส่งค่าจริงเป็น argument ถัดไป เช่น .uri("/products/{id}", id) — Spring จะแทนค่า id ลงในตำแหน่ง {id} ให้อัตโนมัติ (ปลอดภัยกว่าการต่อ String เอง เพราะมีการ encode ค่าให้)

3.3 .retrieve()

เป็นจุดที่บอกให้ WebClient เริ่มส่ง request จริงและเตรียมอ่าน response กลับมา ถ้า response เป็น error status (4xx/5xx) retrieve() จะทำให้ Mono/Flux ที่ตามมาจบด้วย error โดยอัตโนมัติ ซึ่งสามารถดักจับต่อด้วย .onStatus(...) ได้หากต้องการจัดการ error เอง

3.4 .bodyToMono() / .bodyToFlux()

แปลง response body (โดยทั่วไปเป็น JSON) ให้เป็น Java object — ใช้ .bodyToMono(T.class) เมื่อคาดว่า response เป็นวัตถุเดียว (เช่น ดึงสินค้า 1 ชิ้น หรือ ราคาตัวเลขเดียว) และใช้ .bodyToFlux(T.class) เมื่อคาดว่า response เป็น array/list ของวัตถุหลายชิ้น (เช่น ดึงสินค้าทั้งหมด) หลักการเลือกจึงสอดคล้องกับหัวข้อ Mono vs Flux ในข้อ 2

4. Code พร้อมคำอธิบาย

4.1 ProductRepository.java

```
// เก็บข้อมูลจำลองแบบ in-memory ด้วย ConcurrentHashMap
private final Map<String, Product> store = new ConcurrentHashMap<>();

// ค้นหาสินค้า 1 ชิ้นตาม id — เจอคืน Mono.just, ไม่เจอคืน Mono.empty (404)
public Mono<Product> findById(String id) {
    Product product = store.get(id);
    return product != null ? Mono.just(product) : Mono.empty();
}

// คืนสินค้าทั้งหมดในรูปแบบ Flux เพราะมีได้หลายชิ้น
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

// บันทึก/อัปเดตสินค้าใน store แล้วคืนสินค้าที่บันทึกกลับไป
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

// ลบสินค้าตาม id ไม่มีข้อมูลคืนกลับ จึงใช้ Mono<Void>
public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty();
}

// กรองสินค้าตาม category จาก findAll() แล้ว filter ต่อ
public Flux<Product> findByCategory(String category) {
    return findAll().filter(p -> p.getCategory().equalsIgnoreCase(category));
}
```

4.2 ProductService.java

```
// Service ทำหน้าที่เป็นตัวกลาง เรียกใช้ Repository และใส่ business logic เพิ่มเติม
private final ProductRepository repository;

public Mono<Product> getById(String id) {
    return repository.findById(id);
}

public Flux<Product> getAll() {
    return repository.findAll();
}

public Mono<Product> save(Product product) {
    return repository.save(product);
}

public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// ค้นหาสินค้าก่อน แล้ว map เป็นราคาหลังหักส่วนลด
// (logic คำนวณส่วนลดอยู่ใน Product.getDiscountedPrice())
public Mono<Double> getDiscountedPrice(String id) {
    return repository.findById(id)
        .map(Product::getDiscountedPrice)
        .defaultIfEmpty(0.0);
}
```

4.3 ProductController.java

```
// Controller รับ HTTP request แล้วส่งต่อให้ Service เท่านั้น ไม่มี business logic เอง
private final ProductService service;
```

```
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}
```

```
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}
```

```
// รับ Product จาก JSON request body แล้วบันทึก
```

```
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}
```

```
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}
```

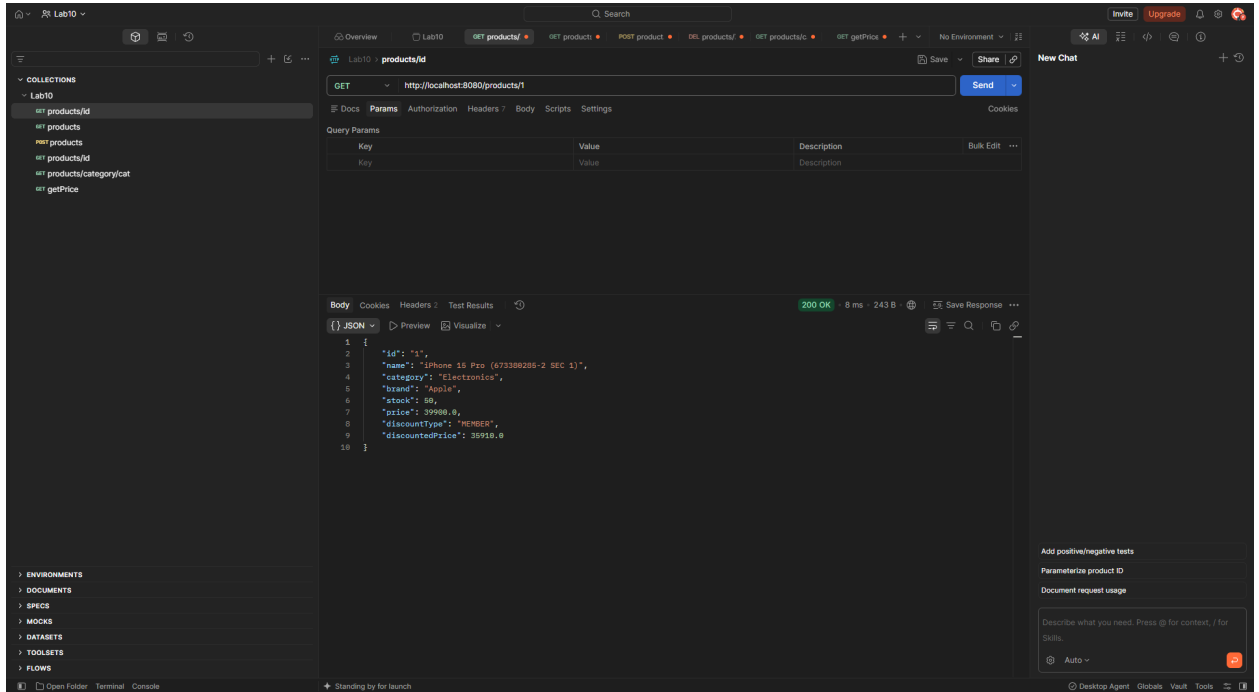
```
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}
```

```
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
```

5. ผลการทดสอบ (Screenshots)

แนบภาพ screenshot การทดสอบแต่ละ endpoint ผ่าน Browser หรือ Postman ต่อจากหัวข้อนี้:

- GET /products/{id}



- GET /products

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/products`. The response is a 200 OK status with a JSON array of three product objects. The left sidebar shows the 'products' collection. The right sidebar shows the 'New Chat' section.

```
1 {
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (473386208-2 SEC 1)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 88,
8     "price": 39999.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35919.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 20,
18    "price": 49999.0,
19    "discountType": "NONE",
20    "discountedPrice": 49999.0
21  },
22  {
23    "id": "3",
24    "name": "Samsung Galaxy S24",
25    "category": "Electronics",
26    "brand": "Samsung",
27    "stock": 30,
28    "price": 29999.0,
29    "discountType": "SEASONAL",
30    "discountedPrice": 24999.0
31  }
32 }
```

- POST /products

The screenshot shows the Postman interface with a POST request to `http://localhost:8080/products`. The request body is a JSON object for a new product. The response is a 200 OK status with the same JSON object. The left sidebar shows the 'products' collection. The right sidebar shows the 'New Chat' section.

```
1 {
2   "id": "4",
3   "name": "Iphone 15 case",
4   "category": "gadget",
5   "brand": "Apple",
6   "stock": 88,
7   "price": 999.0,
8   "discountType": "NONE"
9 }
```

```
1 {
2   "id": "4",
3   "name": "Iphone 15 case",
4   "category": "gadget",
5   "brand": "Apple",
6   "stock": 88,
7   "price": 999.0,
8   "discountType": "NONE",
9   "discountedPrice": 999.0
10 }
```

- DELETE /products/{id}

The screenshot shows the Postman interface with a DELETE request configured. The URL is `http://localhost:8080/products/2`. The response status is `200 OK` with a response time of 3 ms and 38 B of data. The response body is empty.

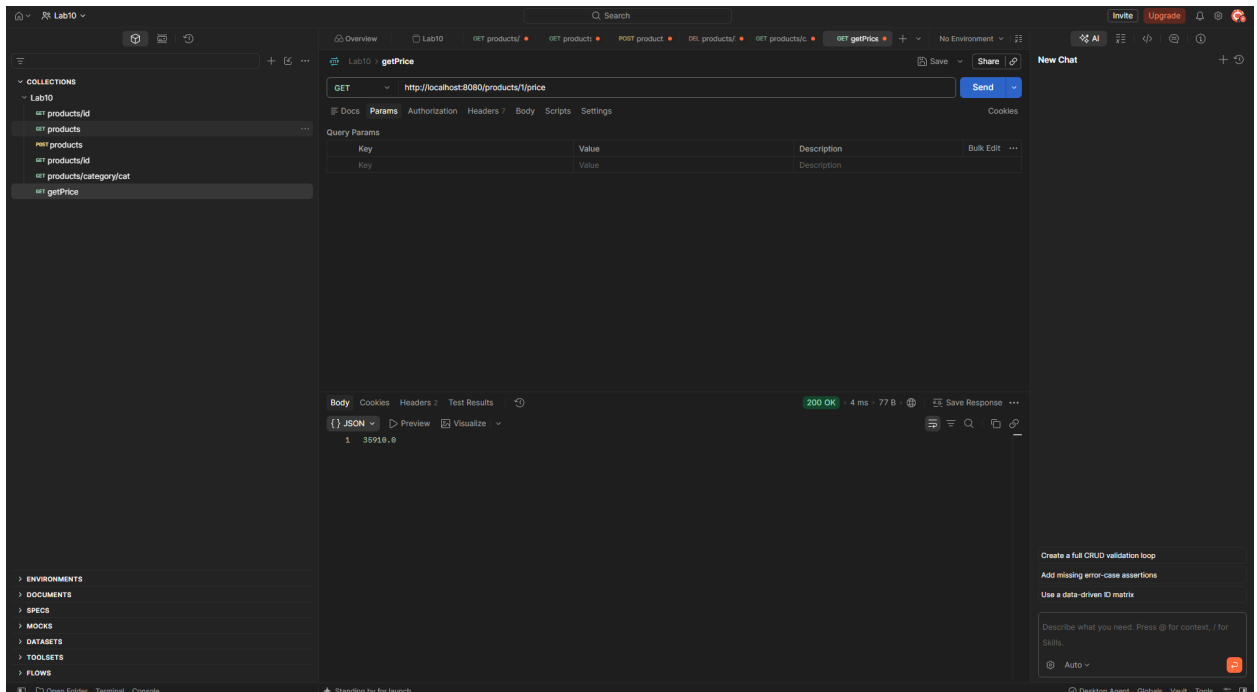
Key	Value	Description	Bulk Edit
Key	Value	Description	

- GET /products/category/{category}

The screenshot shows the Postman interface with a GET request configured. The URL is `http://localhost:8080/products/category/Electronics`. The response status is `200 OK` with a response time of 5 ms and 413 B of data. The response body is a JSON array of two product objects.

```
1 {
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (67308208-2 DEC 1)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 50,
8     "price": 39999.9,
9     "discountType": "MEMBER",
10    "discountedPrice": 36999.9
11  },
12  {
13    "id": "2",
14    "name": "Samsung Galaxy S24",
15    "category": "Electronics",
16    "brand": "Samsung",
17    "stock": 30,
18    "price": 29999.9,
19    "discountType": "SEASONAL",
20    "discountedPrice": 27999.9
21  }
22 }
```

- GET /products/{id}/price



6. สรุป

จาก Lab นี้ได้เรียนรู้การพัฒนา REST API แบบ Reactive ด้วย Spring WebFlux โดยใช้ Mono และ Flux แทนการคืนค่าแบบ blocking ทำให้ระบบสามารถรองรับ concurrent request ได้ดีขึ้นโดยไม่ต้องเพิ่มจำนวน thread นอกจากนี้ยังได้ฝึกใช้ WebClient เป็น reactive HTTP client เพื่อเรียก API ของตัวเองแบบ non-blocking ผ่าน method chain `.get().post().delete()` ร่วมกับ `.uri()`, `.retrieve()` และ `.bodyToMono().bodyToFlux()`